Branch: MCA (Data Science)	**Semester:** 2
Student Name: Adarsh Mishra	**UID:** 25MCD10065
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD KAR-1	**Date of Performance:** 04 April,2026

EXPERIMENT – 08

Aim

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

Tools Used

- PostgreSQL

Objectives

Apply stored procedure concepts for database operations

Theory

In relational databases, stored procedures are used to perform predefined operations on data efficiently. A stored procedure is a set of SQL statements that is stored in the database and can be executed whenever required.

Stored procedures help in reducing code repetition, improving performance, and enhancing security by limiting direct access to tables.

In PostgreSQL, stored procedures are created using `CREATE PROCEDURE` and are mainly used for operations like INSERT, UPDATE, and DELETE. For retrieving data, functions are used with `CREATE FUNCTION`.

Types of Stored Program Units:

• **Procedure**
Used to perform operations such as inserting, updating, and deleting data.

• **Function**
Used to return data from the database using SELECT statements.

• **Parameterized Procedure**
Accepts input parameters to perform dynamic operations.

• **Reusable Stored Code**
Stored procedures can be reused multiple times without rewriting SQL queries.

.Experiment Steps:

Step 1: Create Table

• Create an Employee table
• Define columns: emp_id, name, department, salary
• Set emp_id as Primary Key

Step 2: Insert Data

• Insert sample records into the Employee table
• Ensure data is available for testing operations

Step 3: Create INSERT Procedure

• Create a stored procedure to insert new employee records
• Pass parameters like name, department, and salary

Step 4: Execute INSERT Procedure

• Call the procedure to add a new employee
• Verify the record is inserted

Step 5: Create UPDATE Procedure

• Create a stored procedure to update employee salary
• Use emp_id as input parameter

Step 6: Execute UPDATE Procedure

• Call the procedure to modify employee salary
• Verify the updated value

Step 7: Create DELETE Procedure

• Create a stored procedure to delete an employee
• Use emp_id as input parameter

Step 8: Execute DELETE Procedure

• Call the procedure to remove a record
• Verify deletion from the table

Step 9: Create SELECT Function (ALL)

• Create a function to retrieve all employee records
• Use RETURN QUERY to return table data

Step 10: Create SELECT Function (BY ID)

• Create a function to retrieve a specific employee
• Pass emp_id as parameter

Step 11: Execute SELECT Functions

• Use SELECT statement to call functions
• Display retrieved data

Step 12: Verify Results

• Check final table data
• Ensure all operations (INSERT, UPDATE, DELETE, SELECT) are working correctly

Query:

```
CREATE TABLE Employee2 (
   emp_id SERIAL PRIMARY KEY,
   name VARCHAR(50),
   department VARCHAR(50),
   salary INT
);
```

```
INSERT INTO Employee2 (name, department, salary) VALUES
('Adarsh', 'IT', 50000),
('Kunal', 'HR', 40000),
('Sameer', 'Finance', 60000),
('Chaitanya' , 'Support' , 45000);

CREATE OR REPLACE PROCEDURE insert_employee(
   p_name VARCHAR,
   p_department VARCHAR,
   p_salary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   INSERT INTO Employee2(name, department, salary)
   VALUES(p_name, p_department, p_salary);
END;
$$;



CREATE OR REPLACE PROCEDURE update_employee_salary(
   p_id INT,
   p_salary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   UPDATE Employee2
   SET salary = p_salary
   WHERE emp_id = p_id;
END;
$$;


CREATE OR REPLACE PROCEDURE delete_employee(
   p_id INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   DELETE FROM Employee2
   WHERE emp_id = p_id;
END;
$$;
```

```
CREATE OR REPLACE FUNCTION get_all_employees()
RETURNS TABLE(
   emp_id INT,
   name VARCHAR,
   department VARCHAR,
   salary INT
)
LANGUAGE plpgsql
AS $$
BEGIN
   RETURN QUERY
   SELECT * FROM Employee2;
END;
$$;
```

-- Inserting Employee

```
CALL insert_employee('Ritesh', 'Tech Support', 55000);
```

Output:

```
CALL

Query returned successfully in 137 msec.
```

-- Updating Employee Salary

```
CALL update_employee_salary(1, 70000);
```

Output:

```
CALL

Query returned successfully in 137 msec.
```

--Deleting Employee by ID
CALL delete_employee(2);

Output:

```
CALL

Query returned successfully in 137 msec.
```

-- View Updated Table
SELECT * FROM get_all_employees();

Output:

	emp_id integer	name character varying	department character varying	salary integer
1	3	Sameer	Finance	60000
2	4	Chaitanya	Support	45000
3	5	Ritesh	Tech Support	55000
4	1	Adarsh	IT	70000

Learning Outcomes:

- Understand the concept of **relational databases** and table relationships.
- Identify and differentiate between various types of **SQL JOIN operations** such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and CROSS JOIN.
- Apply JOIN operations to retrieve data from **multiple related tables**.
- Use **foreign keys and primary keys** to establish relationships between tables.
- Analyze query results to understand how different JOINs affect output.
- Implement **multiple table joins** to solve real-world database problems.
- Interpret and validate query outputs for correctness.